Topic 24 — One-Shot Prompting for Deterministic Inference

Full Stack Python + AI Roadmap • Phase 3: Advanced Prompt Engineering • 3 layers: New → Intermediate → Expert

🌱 Layer 1 — New to the Concept

Two ideas combine here:

- **One-shot** = giving the model **one example** to copy.
- **Deterministic inference** = making the model produce the **same output every time** for the same input — no randomness.

Together: *"Here's exactly one example of the format I want, now do it the same way, every single time."*

For creative writing you *want* variety. But for a system that extracts a phone number or classifies a support ticket, you want **boring, predictable, identical** results. Randomness is a bug, not a feature, when the output feeds into other code. The everyday version: a rubber stamp — every stamp comes out identical from one clear template.

👩‍💻 Layer 2 — The Two Levers

Lever 1 — `temperature=0` (the determinism control)

```
response = client.chat.completions.create(
    model="gpt-4",
    messages=messages,
    temperature=0,     # ← the key setting for deterministic output
)
```

- `temperature=0` → model (nearly) always picks the most probable next token → same input gives (nearly) the same output.
- **higher (0.7–1.0)** → more random sampling → varied, creative output.

⚠️ **Note the "nearly":** even at 0, LLMs aren't 100% guaranteed identical every time due to hardware/floating-point factors — but it's as close as you get. Often paired with `top_p=1`.

Lever 2 — the one-shot example (the format anchor)

The single example **removes ambiguity about the output shape**. If the model knows the exact format from your example, it has far less room to vary.

```
prompt = """Extract the phone number from the message. Output ONLY the digits.

Message: "Hey, call me on +91 98765 43210 tomorrow"
```

```
Output: 919876543210

Message: "You can reach me at 022-4455-6677 after 5pm"
Output:"""
```

The example pins down *exactly* what "extract the phone number" means (just digits, no spaces, no `+`). Without it, the model might return `+91 98765 43210` one time and `9876543210` another.

ℹ️ **The combination = maximum reliability:**

```
one-shot example  →  removes format ambiguity   (deterministic shape)
temperature=0     →  removes sampling randomness (deterministic choice)
──────────────────────────────────────────────────────────────
= the same, correctly-formatted output every time
```

🧠 Layer 3 — Expert (production patterns & pitfalls)

1. Canonical use case — structured extraction into your code

```
prompt = """Extract order details as JSON. Follow this exact format.

Text: "I want 3 red shirts and 2 blue jeans"
Output: {"items": [{"qty": 3, "product": "red shirt"}, {"qty": 2, "product": "blue jeans"}]}

Text: "Send me 5 black hoodies please"
Output:"""

# temperature=0 → parseable, consistent JSON every call
response = call_llm(prompt, temperature=0)
data = json.loads(response)     # reliable: format pinned + deterministic
```

The one-shot example shows the exact JSON schema; `temperature=0` ensures it comes back the same way so `json.loads()` doesn't randomly break. (Recall Pydantic from Topic 19 — validate this output as a guardrail.)

2. Why one-shot specifically for deterministic tasks

- **Zero-shot** leaves format ambiguity → model may vary presentation → less deterministic in practice.
- **One-shot** is often the **sweet spot**: one example kills ambiguity with minimal token cost.
- **Few-shot** adds more examples — helps for *pattern* variety, but for a *single fixed format* the extra examples are often unnecessary tokens.

✅ **Default:** for "same format, every time" tasks, one-shot + `temperature=0` is the efficient choice.

3. Determinism isn't only temperature — the `seed` parameter

```python
response = client.chat.completions.create(
    model="gpt-4",
    messages=messages,
    temperature=0,
    seed=42,           # request reproducible sampling
)
```

A fixed `seed` + `temperature=0` gets you as close to true reproducibility as possible — useful for testing, debugging, and caching. (Still best-effort, not a hard guarantee across model versions.)

4. Caching — a bonus of determinism

Because the same input reliably gives the same output, you can **cache** results: hash the prompt, store the response, and skip the API call (and cost) next time the identical input appears. Determinism makes caching safe — with high temperature, caching would serve one random variant forever. A real cost-saver at scale (relevant to MSG2AI's repetitive messages).

5. The pitfalls

🚨 **Watch out:**

- `temperature=0` **≠ correct.** Deterministic means *consistent*, not *right*. A wrong answer at temp 0 is wrong every time. Still need good prompts + validation.
- **Not guaranteed across model updates.** Provider updates can shift outputs. Pin model versions (e.g. `gpt-4-0613`) if exact reproducibility matters.
- **Over-constraining.** Too narrow an example → model copies it too literally and fails on differing inputs. Choose a *representative* example.
- **Still validate.** Even deterministic output can be malformed on edge cases — parse/validate (Pydantic), don't trust blindly.

6. When you do NOT want determinism

Flip the lever for tasks where variety is the goal — brainstorming, creative writing, generating multiple options, paraphrasing — by *raising* temperature. The skill is matching setting to task: **low temp for extraction/classification/structured output; higher temp for creative/generative work.**

🎯 Interview Q&A Cards

Q: What is deterministic inference and how do you get it?

A: Getting the same output for the same input, achieved mainly with `temperature=0` (optionally a fixed `seed`). Pairing it with a one-shot example removes format ambiguity, yielding consistently-shaped, parseable output.

Q: Why combine one-shot with temperature=0?

A: The one-shot example pins the output *shape*; `temperature=0` pins the token *choice*. Together they give the same correctly-formatted output every time — ideal for extraction/classification feeding into code.

Q: Does temperature=0 guarantee correct answers?

A: No — it guarantees *consistent*, not *correct*. A wrong answer becomes wrong every time. You still need good prompts, output validation, and pinned model versions for true reproducibility.

Q: What does determinism enable operationally?

A: Safe caching (hash prompt → reuse stored response, skipping cost) and reliable testing/debugging, because identical inputs reliably produce identical outputs.

🪝 **Memory hook:** *One-shot pins the format; `temperature=0` pins the choice → same output every time. The combo is the default for extraction/classification into code. Enables caching. But deterministic ≠ correct — still validate, still pin model versions. Raise temperature only when you want variety.*